Change Control: Prompt and Model Changes

Process document C(ii). Covers any change to a prompt in `copilot/prompts/` or `eval/judge_v1.md`, to `COPILOT_RUNTIME_MODEL` or `COPILOT_JUDGE_MODEL`, and to the retrieval settings that change what the model sees (`TOP_K`, `FALLBACK_MARGIN`, chunking). Roles as in the SOP: **Associate** (usually the proposer), **Lead** (approver), **Support Lead** (SunGrid sign-off on member-facing wording), **Backend engineer** (release and rollback). Commands run from the repository root with the virtualenv active.

Why these are controlled. Prompts and models are code that nobody can read the behaviour of. On this system, one prompt line removed or one model swapped can silently turn the hard gate off or break citations. The first end-to-end run showed the second: a formatting habit of the model gave false "not found" answers on 2 questions.

1. Change classes

Class	Examples	Approval	Release
A: wording only	Fix a typo in a refusal or template	Lead	Normal release after tests
B: behaviour	Edit any prompt, change `TOP_K` or `FALLBACK_MARGIN`, change chunking	Lead and Support Lead	Full procedure below
C: model	New runtime or judge model, new model version, new sampling settings	Lead and Support Lead	Full procedure, plus cost check and canary stage
D: gate	Anything in `copilot/tools/eligibility.py` validation or templates, or `stub_tools.py` wiring	Lead and the eligibility service owner	Full procedure; the gate tests must not be edited in the same change

When in doubt, treat a change as the higher class. `stub_tools.py` itself is owned by another team and is never edited here; `tests/test_stub_unchanged.py` fails if it is.

2. Procedure

1. **Propose.** Open a ticket stating: what changes, why (the failing trace or question that motivated it), the expected effect, and the expected risk. Attach the trace ids that show the problem. One concern per change.
2. **Branch and edit.** Prompts are files, versioned in Git. For a class B or C change, copy the prompt to a new name (`classifier_v2.md`) and point the code at it, so the old one stays on disk for rollback. Never edit a prompt in a deployed environment.
3. **Test, offline.** Run `pytest` (no API calls, must pass in full). Then run the live evals against the **baseline report committed on `main`**: `python -m eval.run_eval retrieval`, `classifier`, `eligibility`, `e2e` (3 runs each, roughly $0.12 for all). Keep the new reports; do not overwrite the baseline until merge.
4. **Compare.** The proposer writes a table of old versus new for every suite and lists **every question whose result changed**, in either direction. Look at the answers, not only the counts: the judge was only spot-checked, so read at least 5 changed answers by hand.
5. **Gate checks (a failure blocks the change).**
6. Eligibility gate respected on every case, 100%.
7. No dollar figure in any non-eligible reply.
8. No question that passed before now fails, unless the Lead accepts it in writing.
9. Classifier category and intent not lower than baseline.
10. Retrieval recall@5 not lower than baseline.
11. End-to-end pass rate not lower than baseline (66 of 69 now), with no new contradictions.
12. Median cost and latency within 20% of baseline. A model change that raises cost per query more than 20% needs a cost note to the budget owner. Scores are from a small set written by one person, so a pass is evidence, not proof. For a class C change, also add 5 real questions from recent traces before running, so the set is not just the questions the old prompt was tuned on.
13. **Approve.** Approvers in section 1 comment "approved" on the ticket after reading the comparison table and the changed answers. The proposer cannot approve their own change.
14. **Release.** Merge to `main` and deploy with the normal pipeline. For class C changes, route 5% of traffic to the new version first (canary) for 48 hours, then 100%. Record the release in the changelog: date, ticket, prompt or model id before and after, report file names, approver names.
15. **Monitor.** For 7 days after release, check the dashboard daily: abstain rate, fallback rate, rejected-citation rate, eligibility outcome split, p95 latency, cost per query, thumbs-down count. Compare with the 4-week average. Review all thumbs-down traces and a 5% sample. **Dollar figure in a non-eligible reply: roll back immediately.**
16. **Close.** On day 7 the proposer posts the monitoring summary. If any real failing question was found, add it to `eval/questions.yaml`, so the next change is tested against it.

3. Rollback

- **Trigger (any one):** gate violation; abstain rate or rejected-citation rate more than double the 4-week average for a day; p95 latency doubled; cost per query over 1.5x the baseline; a Support Lead report of a wrong answer given to a member.

- **Who:** anyone on the team can start it; they do not need approval first. The Backend engineer or Lead does it.
- **How:** point the code or `COPILOT_RUNTIME_MODEL` back to the previous prompt file or model id (both are still in the repository or environment), redeploy, and confirm with the smoke test (one question per path: knowledge, eligibility eligible, eligibility ineligible, non-relevant). Target: under 30 minutes.
- **After:** a short written note within 2 working days: what happened, which check should have caught it, what is added to the eval set.
- **Model retired by the provider:** treat as a class C change, with a deadline. Keep a note of the provider's announced retirement dates, review it monthly.

4. Release checklist

Copy into every class B, C and D ticket and tick each line.

- [ ] One concern per change; ticket states reason, expected effect and risk
- [ ] Old prompt kept on disk under its old name (class B, C)
- [ ] `pytest` passes in full, `tests/test_stub_unchanged.py` included
- [ ] All four eval suites run; report files saved and committed
- [ ] Old versus new table written; every changed question listed; 5 changed answers read by hand
- [ ] Gate checks in section 2, step 5 all met, or the exception written and accepted by the Lead
- [ ] Class C: 5 recent real questions added; cost per 1,000 queries recomputed
- [ ] No API key, trace or member data in the diff
- [ ] Approvals recorded (Lead; Support Lead for B and C; service owner for D)
- [ ] Rollback target named (file or model id) and smoke test ready
- [ ] Canary period done (class C)
- [ ] Changelog entry written
- [ ] 7-day monitoring summary posted, new failing cases added to the eval set

5. Known gaps (to be fixed, and why they matter)

- The pass thresholds are my starting values, not measured limits, and the evals are small (20 questions plus probes, one author), so a change can pass and still be worse. Grow the set from real traces every release.
- The LLM judge is unvalidated beyond a spot check. Until staff labels are collected, read changed answers by hand.
- Runs cannot be seeded (the runtime model rejects sampling settings), so results vary between runs. The 3-run rule exists for this; treat a one-question difference as noise unless it repeats.